Technology Stacks:

1. python:

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ python --version
Python 3.14.4
```

2. github:

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ git --version
git version 2.53.0
```

3. docker:

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ docker --version
docker run --rm hello-world
Docker version 29.8.0, build 88096ef

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

4. NumPy + Matplotlib:

stack_test.py

```
import sys
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

print("=== TECHNOLOGY STACK TEST ===")
print("Python:", sys.version.split()[0])
print("NumPy:", np.__version__)
print("Matplotlib:", matplotlib.__version__)

# Experiment data
epochs = np.arange(1, 11)
loss = np.array([
    0.95, 0.78, 0.64, 0.53, 0.44,
    0.37, 0.31, 0.27, 0.24, 0.21
])

# Basic numerical analysis
print("Number of epochs:", len(epochs))
print("Initial loss:", loss[0])
print("Final loss:", loss[-1])
print("Loss reduction:", round(loss[0] - loss[-1], 3))

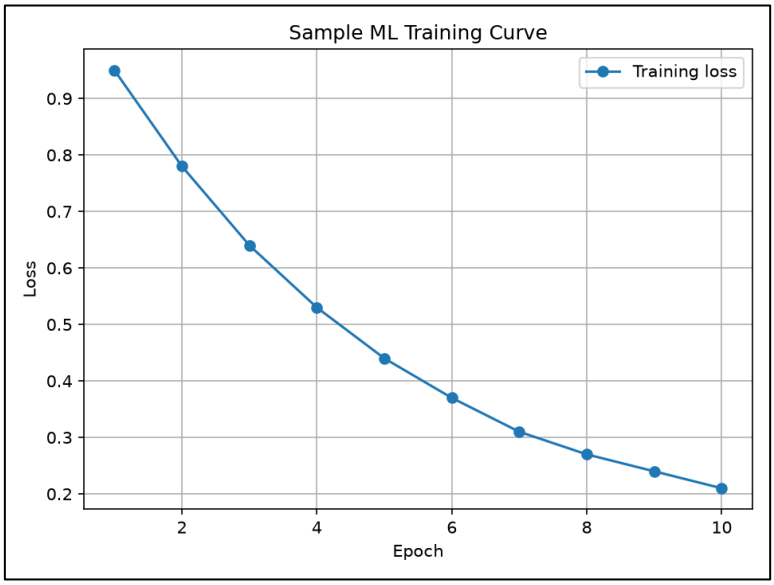
# Generate and save a plot
plt.plot(epochs, loss, marker="o", label="Training loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
```

```
plt.title("Sample ML Training Curve")
plt.grid(True)
plt.legend()
plt.tight_layout()

plt.savefig("ml/training_loss.png", dpi=150)
print("Plot saved: ml/training_loss.png")

plt.show()
```

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ python3 ml/stack_tes
t.py
=== TECHNOLOGY STACK TEST ===
Python: 3.14.4
NumPy: 2.5.4
Matplotlib: 3.11.2
Number of epochs: 10
Initial loss: 0.95
Final loss: 0.21
Loss reduction: 0.74
Plot saved: ml/training_loss.png
```



5. C/C++ compiler:

hello.cpp

```
#include <iostream>

int main() {
    int a = 10;
    int b = 20;

    std::cout << "C++ stack test\n";
    std::cout << "a + b = " << a + b << '\n';

    return 0;
}
```

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ g++ --version
g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ g++ cpp/hello.cpp -o
 cpp/hello
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ ./cpp/hello
C++ stack test
a + b = 30
```

6. scikit-learn:

sklearn_test.py

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import numpy as np

# Two features per sample
X = np.array([
    [1.0, 1.0],
    [1.5, 1.0],
    [2.0, 1.5],
    [7.0, 7.0],
    [8.0, 7.0],
    [9.0, 8.0]
])

y = np.array([0, 0, 0, 1, 1, 1])

model = LogisticRegression()
model.fit(X, y)

predictions = model.predict(X)
probabilities = model.predict_proba(X)

print("Predictions:", predictions)
print("Training accuracy:", accuracy_score(y, predictions))
print("First sample probabilities:", probabilities[0])
```

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ python3 ml/sklearn_test.py
Predictions: [0 0 0 1 1 1]
Training accuracy: 1.0
First sample probabilities: [0.97922115 0.02077885]
```

7. PyTorch:

pytorch_test.py

```
import torch

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

# y = wx + b
x = torch.tensor([1.0, 2.0, 3.0])
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)

y = w * x + b
loss = y.mean()

loss.backward()

print("Predicted values:", y.detach().tolist())
print("Mean output:", loss.item())
print("Gradient of w:", w.grad.item())
print("Gradient of b:", b.grad.item())
```

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ python ml/pytorch_test.py
PyTorch: 2.14.1+cu130
CUDA available: False
Predicted values: [3.0, 5.0, 7.0]
Mean output: 5.0
Gradient of w: 2.0
Gradient of b: 1.0
```

8. NetworkX and OR-Tools:

graph_optimization_test.py

```
import networkx as nx
```

```
from ortools.sat.python import cp_model

# Part A: a simple circuit dependency graph
graph = nx.DiGraph()
graph.add_edges_from([
    ("A", "N1"),
    ("B", "N1"),
    ("N1", "OUT")
])

print("Graph nodes:", list(graph.nodes()))
print("Graph edges:", list(graph.edges()))
print("Is acyclic:", nx.is_directed_acyclic_graph(graph))

# Part B: a tiny integer optimization problem
model = cp_model.CpModel()

x = model.new_int_var(0, 10, "x")
y = model.new_int_var(0, 10, "y")

model.add(x + y <= 10)
model.maximize(3 * x + 2 * y)

solver = cp_model.CpSolver()
status = solver.solve(model)

if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    print("x =", solver.value(x))
    print("y =", solver.value(y))
    print("Objective =", solver.objective_value)
else:
    print("No feasible solution found.")
```

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ python eda/graph_opt
imization_test.py
Graph nodes: ['A', 'N1', 'B', 'OUT']
Graph edges: [('A', 'N1'), ('N1', 'OUT'), ('B', 'N1')]
Is acyclic: True
x = 10
y = 0
Objective = 30.0
```

9. TCL:

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ tclsh
% puts "TCL technology stack test"

set width 8
set depth 256

puts "Width = $width"
puts "Depth = $depth"
puts "Capacity in bits = [expr {$width * $depth}]"
exitTCL technology stack test
% % 8
% 256
% % Width = 8
% Depth = 256
% Capacity in bits = 2048
%
```

10. Pandas:

pandas_lab.py

```
import pandas as pd

# Create a small VLSI timing dataset
data = {
    "Circuit": ["ALU", "FIFO", "UART", "MAC", "ALU"],
    "Delay_ns": [2.5, 3.1, 1.8, 4.2, 2.7],
    "Power_mW": [12.0, 8.5, 3.2, 18.0, 11.5],
    "Timing_Violations": [0, 2, 0, 4, 1]
```

```
}

df = pd.DataFrame(data)

print("Complete dataset:")
print(df)

print("\nAverage delay:")
print(df["Delay_ns"].mean())

print("\nCircuits with delay greater than 2.5 ns:")
print(df[df["Delay_ns"] > 2.5])

print("\nCircuits with timing violations:")
print(df[df["Timing_Violations"] > 0])
```

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$
 python pandas_lab.py
Complete dataset:
  Circuit  Delay_ns  Power_mW  Timing_Violations
0     ALU       2.5      12.0                  0
1    FIFO       3.1       8.5                  2
2    UART       1.8       3.2                  0
3     MAC       4.2      18.0                  4
4     ALU       2.7      11.5                  1

Average delay:
2.8600000000000003

Circuits with delay greater than 2.5 ns:
  Circuit  Delay_ns  Power_mW  Timing_Violations
1    FIFO       3.1       8.5                  2
3     MAC       4.2      18.0                  4
4     ALU       2.7      11.5                  1

Circuits with timing violations:
  Circuit  Delay_ns  Power_mW  Timing_Violations
1    FIFO       3.1       8.5                  2
3     MAC       4.2      18.0                  4
4     ALU       2.7      11.5                  1
```

11. Verilog and SystemVerilog toolchain:

and_gate.sv

```
module and_gate (
    input  logic a,
    input  logic b,
    output logic y
);
    assign y = a & b;
endmodule
```

tb_and_gate.sv

```
`timescale 1ns/1ps

module tb_and_gate;
    logic a;
    logic b;
    logic y;

    and_gate dut (
        .a(a),
        .b(b),
        .y(y)
    );

    initial begin
        $dumpfile("rtl/and_gate.vcd");
        $dumpvars(0, tb_and_gate);

        $display("time  a b | y");
        $monitor("%4t   %0b %0b | %0b", $time, a, b, y);

        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

        $display("Simulation complete.");
        $finish;
```

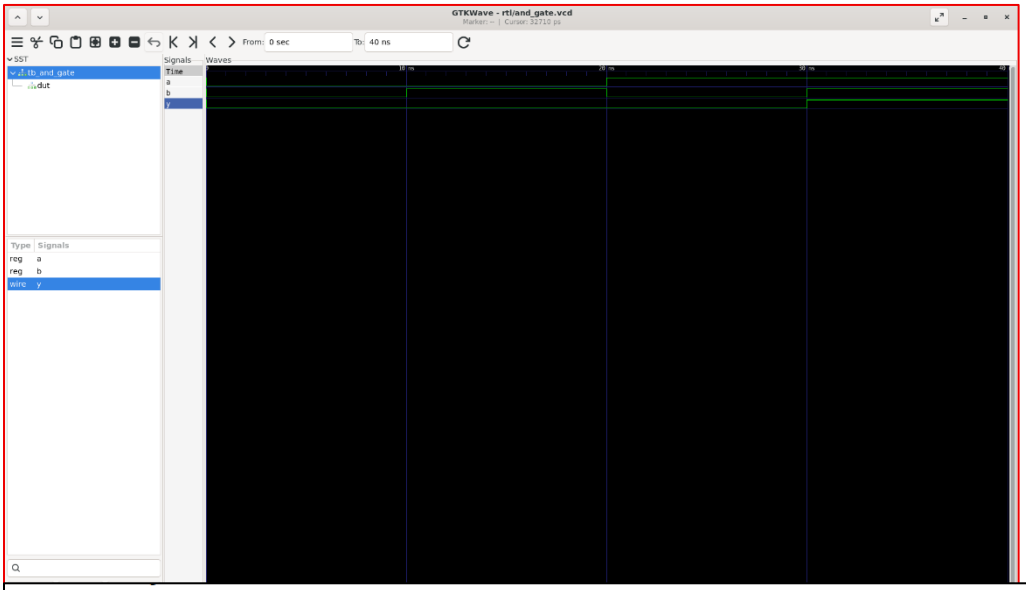
```
    end
endmodule
```

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-st
ack-lab$ iverilog -g2012 -o rtl/and_gate.out \
    rtl/and_gate.sv rtl/tb_and_gate.sv

vvp rtl/and_gate.out
VCD info: dumpfile rtl/and_gate.vcd opened for output.
time  a b | y
   0   0 0 | 0
10000   0 1 | 0
20000   1 0 | 0
30000   1 1 | 1
Simulation complete.
rtl/tb_and_gate.sv:28: $finish called at 40000 (1ps)
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-st
ack-lab$ gtkwave rtl/and_gate.vcd

GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
[40000] end time.
```




```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-st
ack-lab$ yosys -p "read_verilog -sv rtl/and_gate.sv; hierarchy -top and_gate; pro
c; opt; check; stat"
```

```
=== and_gate ===

   Number of wires:                 3
   Number of wire bits:             3
   Number of public wires:          3
   Number of public wire bits:      3
   Number of ports:                 3
   Number of port bits:             3
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 1
     $and                           1
```

i. Icarus Verilog

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ iverilog -V
Icarus Verilog version 12.0 (stable) ()
```

ii. GTKWave

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ gtkwave --version
GTKWave Analyzer v3.3.126 (w)1999-2026 BSI
```

iii. Yosys

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ yosys -V
Yosys 0.52 (git sha1 fee39a3284c90249e1d9684cf6944ffbbcbb8f90)
```

iv. Verilator

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ verilator --version
Verilator 5.032 2025-01-01 rev (Debian 5.032-1)
```

v. OpenSTA

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ sta -version
2.0.17
```

vi. OpenROAD

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ docker run --r
m openroad/orfs:latest bash -lc 'source ./env.sh && openroad -version && yosys -V'
OPENROAD: /OpenROAD-flow-scripts/tools/OpenROAD
26Q3-2056-g41a28926b9
Yosys 0.68+post (git sha1 UNKNOWN, Release, GNU /usr/local/bin/wrapped-cc/g++ 11.4.0)
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ docker run --r
m -it openroad/orfs:latest bash
root@ac098d1fec02:/OpenROAD-flow-scripts# source ./env.sh
openroad -version
yosys -V
OPENROAD: /OpenROAD-flow-scripts/tools/OpenROAD
26Q3-2056-g41a28926b9
Yosys 0.68+post (git sha1 UNKNOWN, Release, GNU /usr/local/bin/wrapped-cc/g++ 11.4.0)
root@ac098d1fec02:/OpenROAD-flow-scripts#
```

vii. KLayout

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$ klayout -v
KLayout 0.30.0
```

viii. Magic

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$
 magic --version
8.3.105
```

ix. Netgen

```
(.venv) sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/Learn Python/technology-stack-lab$
 netgen -batch help
NETGEN-6.2.2601
```

x. OpenLane

```
(openlane-venv) root@0535dd400444:/# python -m openlane --version
OpenLane v2.3.10
```